

SIMATS ENGINEERING

ASSESSMENT TOOL 3

COURSE NAME: OBJECT ORIENTED ANALYSIS AND DESIGN
COURSE CODE: CSA1153

COURSE OUTCOME COVERED

CO4:

implement object-oriented system layers using design principles and patterns, and integrate access and view layers with OODBMS (BL5)

Assessment Tool Weightage: 30%

QUESTIONS: 1

Total Marks:10

Annexure A

Reverse Engineering Task

Code of Conduct:

I D RAGHU VARDHAN 192311051 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

D RAGHU VARDHAN

Annexure A

Reverse Engineering Task

1. An existing system is developed without following object-oriented principles, resulting in tightly coupled classes and poor modularity.

Question:

Reverse engineer the system by applying object-oriented design principles (encapsulation, inheritance, polymorphism) and restructure it into a modular design. Explain your approach.

Reverse Engineering a Non-Object-Oriented System Using Object-Oriented Design Principles

Introduction

The existing system was developed without following object-oriented principles, resulting in tightly coupled classes, duplicated code, poor maintainability, and low reusability. Reverse engineering involves analyzing the current system, identifying design issues, and redesigning it using object-oriented concepts. By applying **encapsulation, inheritance, and polymorphism**, the system can be transformed into a modular, flexible, and maintainable architecture.

Problems in the Existing System

The non-object-oriented system has the following drawbacks:

- Tightly coupled classes.
- Duplicate code across modules.
- Direct access to data members.
- Difficult to add new features.
- High maintenance cost.
- Low reusability of components.
- Changes in one module affect other modules.

Step 1: Analyze the Existing System

The first step in reverse engineering is studying the source code and identifying related functionalities.

Activities:

- Examine existing classes and modules.
- Identify repeated code segments.
- Detect dependencies between modules.
- Understand data flow and business processes.
- Identify common behaviors among classes.

This analysis helps determine where object-oriented principles can be introduced.

Step 2: Apply Encapsulation

Encapsulation combines data and methods into a single unit and restricts direct access to internal data.

Before Encapsulation

Customer	Name
Customer	ID
Balance	
All data can be modified directly by any module.	

After Encapsulation

Customer	Class

-	customerId
-	customerName
-	balance
+	getDetails()
+	updateBalance()
+	deposit()
+	withdraw()

Benefits

- Protects data from unauthorized access.
- Improves security.
- Simplifies maintenance.
- Reduces system complexity.

Step 3: Apply Inheritance

Inheritance allows common features to be placed in a parent class and reused by child classes.

Example

Instead of creating separate code for different users:

Customer

Admin

Employee

Create a common parent class:

User

userId

name

login()

|

| Customer Admin Employee |

Benefits

- Eliminates duplicate code.
- Promotes code reuse.
- Simplifies maintenance.
- Reduces development effort.

Step 4: Apply Polymorphism

Polymorphism allows different objects to respond differently to the same method call.

Example

Payment

|

| UPI CreditCard NetBanking |

Each class implements:

processPayment()

Example Output

UPI

Payment

Processed

Credit

Card

Payment

Processed

Net Banking Payment Processed

Benefits

- Increases flexibility.
 - Supports future extensions.
 - Simplifies implementation of new features.
-

Step 5: Modular Design Restructuring

The system is reorganized into independent modules.

Presentation Layer

Responsibilities

- User Interface
- User Input Handling
- Display Results

Examples

- Login Screen
- Dashboard
- Reports

Business Logic Layer

Responsibilities

- Business Rules
- Transaction Processing
- Validation

Examples

- Order Processing
- Payment Handling
- Inventory Checking

Data Access Layer

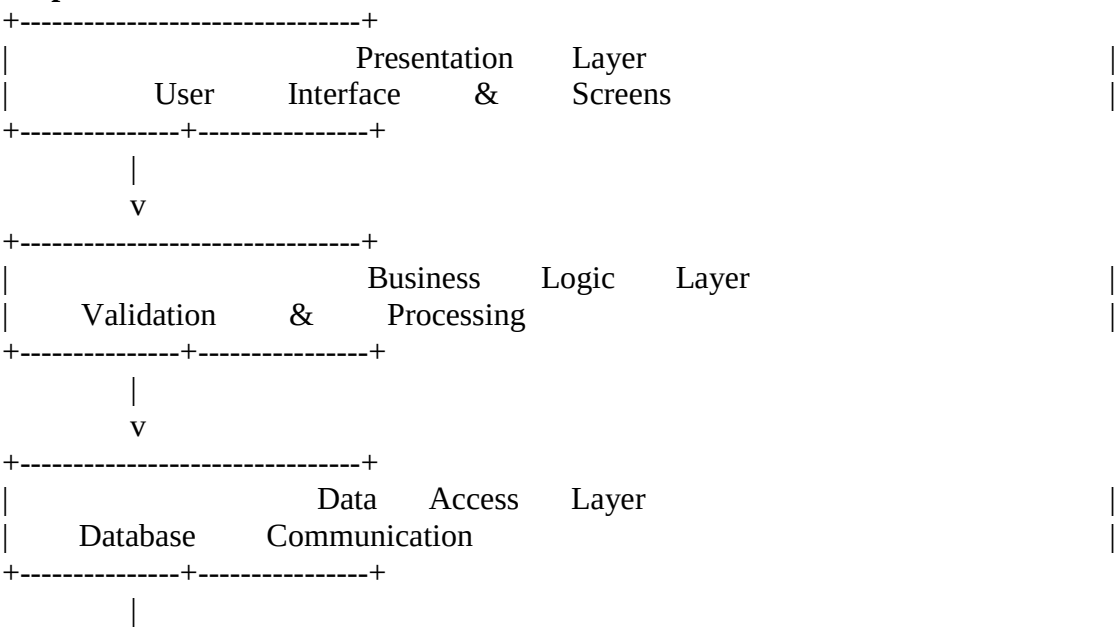
Responsibilities

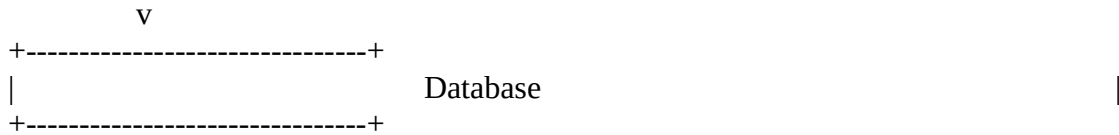
- Database Operations
- Data Storage
- Data Retrieval

Examples

- CustomerDAO
- ProductDAO
- OrderDAO

Proposed Modular Structure





Advantages of the Redesigned System

Improved Maintainability

Changes can be made in one module without affecting others.

Better Reusability

Classes can be reused across multiple applications.

Enhanced Flexibility

New features can be added easily.

Reduced Coupling

Modules interact through well-defined interfaces.

Increased Scalability

The system can grow without major redesign.

Easier Testing

Individual modules can be tested independently.

Reverse Engineering Approach Summary

Phase 1: System Analysis

- Study existing code.
- Identify dependencies and duplicated functionality.

Phase 2: Object Identification

- Create classes and objects.
- Define responsibilities.

Phase 3: Apply OOP Principles

- Encapsulation for data security.
- Inheritance for code reuse.
- Polymorphism for flexibility.

Phase 4: Modularization

- Separate UI, business logic, and database operations.

Phase 5: Testing and Validation

- Verify functionality.
- Ensure performance and maintainability.

Conclusion

By reverse engineering the existing system and applying **encapsulation, inheritance, and polymorphism**, the tightly coupled design is transformed into a modular object-oriented architecture. The new structure improves maintainability, scalability, reusability, and flexibility while reducing code duplication and system complexity. The modular layered design also makes future enhancements easier and more cost-effective, resulting in a robust and efficient software system.

Existing System
(Tightly Coupled Classes)

|

v

Reverse Engineering

|

| | |
Encapsulation Inheritance Polymorphism
| | |

|

v

Modular OOP Design

|

| | |
Presentation Business Data
Layer Layer Access Layer
|
v
Database

RUBRICS FOR CALCULATION

Reverse Engineering Task

Criteria	Weightage	Exemplary (4)	Proficient (3)	Developing (2)	Beginning (1)
Understanding of System	20%	Demonstrates clear and in-depth understanding of the system/product and its functionality	Shows good understanding with minor gaps	Partial understanding; misses key aspects	Lacks understanding of the system/product
Decomposition	25%	Effectively breaks down the system into components with detailed analysis	Decomposes system with minor gaps in analysis	Limited or incomplete decomposition	Unable to analyze or break down system
Identification of Components	20%	Accurately identifies all components and explains their functions clearly	Identifies most components with minor errors	Identifies few components; explanations are weak	Unable to identify components or functions
Reconstruction	20%	Successfully reconstructs or models the system with accuracy and logic	Reconstruction is mostly correct with minor issues	Partial reconstruction; lacks clarity	Unable to reconstruct or model system
Documentation	15%	Well-organized, clear documentation with diagrams and structured explanation	Documentation is adequate with minor issues	Poorly organized or incomplete documentation	No proper documentation or presentation